

ROS2 런치 파일 & 파라미터 관리

Launch File · Parameter · YAML

cchyun@gmail.com

Part 1. 런치 파일(Launch File) 기초

런치 파일이 왜 필요한가?

- 로봇 시스템은 수많은 노드로 구성 → 하나하나 수동 실행은 한계
 - 터미널 관리의 한계
 - 노드가 10개, 20개로 늘어날 때마다 매번 새 터미널에서 `ros2 run` 입력
 - 비효율적이고 실수가 잦음
 - 단일 명령어로 전체 시스템 기동
 - 런치 파일 = 로봇 시스템 전체 또는 특정 서브시스템을 **단일 명령어**로 실행
 - 예: 센서부, 구동부, 인식부를 한 번에 기동
 - 설정 자동화 및 재사용성 향상
 - 여러 노드의 파라미터, 네임스페이스, 토픽 리매핑을 하나의 파일에 기술
 - 시스템 구성 표준화 → 팀원 간 공유·재사용 가능

- ROS2는 XML, YAML, Python 세 가지 런치 파일 형식을 지원
 - XML / YAML
 - 단순한 노드 실행에는 충분
 - 복잡한 조건 처리나 동적 구성에는 한계
 - Python (권장)
 - 조건문(`if`), 반복문(`for`), 변수, 함수 등 Python 문법 완전 활용 가능
 - 복잡한 로봇 시스템 구성에 가장 유연하고 강력함
 - ROS2 공식에서 가장 권장하는 방식

Part 2. Python 런치 파일 기본 구조

- 모든 Python 런치 파일의 필수 진입 함수
 - 함수 역할
 - 런치 시스템이 이 함수를 호출하여 실행할 액션 목록을 가져옴
 - 반드시 `LaunchDescription` 객체를 반환해야 함

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        # 실행할 액션(노드 등)들을 여기에 나열
    ])
```

- 기억 포인트
 - 함수 이름은 반드시 `generate_launch_description` 으로 고정
 - `LaunchDescription([액션1, 액션2, ...])` 형태로 반환

Node() 액션 — 핵심 파라미터

- `Node()` = 노드 하나를 실행하는 기본 액션 클래스

- 주요 파라미터 상세 설명

파라미터	설명	예시
<code>package</code>	실행할 노드가 있는 패키지 이름	<code>'usb_cam'</code>
<code>executable</code>	패키지 내 실행 파일(entry_point) 이름	<code>'usb_cam_node_exe'</code>
<code>name</code>	노드 실행 시 강제 지정할 이름	<code>'front_camera'</code>
<code>namespace</code>	토픽/서비스 충돌 방지용 그룹 이름	<code>'robot_a'</code>
<code>remappings</code>	토픽/서비스 이름을 다른 이름으로 연결	<code>[('/in', '/out')]</code>
<code>parameters</code>	노드에 전달할 파라미터 (dict 또는 YAML 경로)	<code>[{'rate': 30.0}]</code>

Node() 액션 — 사용 예시

- Node() 기본 사용 예시

```
from launch_ros.actions import Node

camera_node = Node(
    package='usb_cam',
    executable='usb_cam_node_exe',
    name='raw_camera',           # 노드 이름 강제 지정
    namespace='robot_1',        # 네임스페이스 설정
    remappings=[
        ('/image_raw', '/robot_1/image_raw') # 토픽 리매핑
    ],
    parameters=[{'video_device': '/dev/video0'}] # 파라미터 직접 전달
)
```

- `name` : 동일 노드를 여러 개 실행할 때 중복 방지
- `namespace` : `/robot_1/image_raw` 처럼 그룹화된 토픽 이름 생성
- `remappings` : 기존 토픽을 다른 이름에 연결 (파이프 연결 개념)

자주 쓰이는 런치 액션 종류

- **Node** 외에도 다양한 런치 액션이 존재
 - **Node**
 - 단일 노드를 실행하는 가장 기본적인 액션
 - **IncludeLaunchDescription**
 - 이미 작성된 다른 런치 파일을 불러와 포함시킴
 - 시스템을 모듈화할 때 필수적 — 서브시스템별 런치 파일을 조합
 - **GroupAction** + **PushRosNamespace**
 - 여러 액션을 그룹으로 묶고, 그룹 전체에 네임스페이스를 일괄 부여
 - **TimerAction**
 - 지정된 시간(초)만큼 지연 후 노드 기동
 - 의존 관계가 있는 노드들의 순서 제어에 활용

(실습) 패키지 생성 & 구조 준비

- 워크스페이스 및 패키지 생성

```
# 워크스페이스 생성
cd ~/Workspace/ros2_ws/src/

# Python 패키지 생성
ros2 pkg create --build-type ament_python py_launch_example

# 런치 파일 디렉터리 생성
cd py_launch_example
mkdir launch
```

- 디렉터리 구조

```
src/
  py_launch_example/
    launch/           ← 런치 파일 저장 위치
    package.xml
    py_launch_example/
    resource/
    setup.cfg
    setup.py
    test/
```

(실습) Python 런치 파일 작성

- `launch/example_bringup_launch.py` 작성 (1/2)

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import DeclareLaunchArgument, GroupAction
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import PushRosNamespace
from launch.conditions import IfCondition

def generate_launch_description():
    # 런치 인자 선언
    use_rviz_arg = DeclareLaunchArgument(
        'use_rviz',
        default_value='false',
        description='rqt_image_view 실행 여부'
    )

    # image_publisher
    image_publisher = Node(
        package='camera_pkg',
        executable='img_pub'
    )
```

(실습) Python 런치 파일 작성

- `launch/example_bringup_launch.py` 작성 (2/2)

```
# image_yolo
image_yolo = Node(
    package='camera_pkg',
    executable='img_yolo'
)

# image_edge
img_canny = Node(
    package='camera_pkg',
    executable='img_canny'
)

# rqt_image_view (조건부 실행)
viewer_node = Node(
    package='rqt_image_view',
    executable='rqt_image_view',
    condition=IfCondition(LaunchConfiguration('use_rviz'))
)

return LaunchDescription([
    use_rviz_arg,
    image_publisher,
    image_yolo,
    img_canny,
    viewer_node,
])
```

(실습) setup.py 수정 (설치 경로 등록)

- `setup.py`에 `launch-config` 폴더 등록

```
import os
from glob import glob
from setuptools import find_packages, setup

package_name = 'py_launch_example'

setup(
    name=package_name,
    version='0.0.1',
    packages=find_packages(),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        # ★ 런치 파일 설치 경로
        (os.path.join('share', package_name, 'launch'),
         glob('launch/*.py')),
        # ★ config 파일 설치 경로
        (os.path.join('share', package_name, 'config'),
         glob('config/*.yaml')),
    ],
    ...
)
```

- 빌드 및 런치 파일 실행

```
cd ~/Workspace/ros2_ws  
colcon build  
source install/setup.bash
```

- 기본 실행 (namespace 없음)

```
ros2 launch py_launch_example example_bringup_launch.py
```

- rqt_image_view 함께 실행

```
ros2 launch py_launch_example example_bringup_launch.py use_rviz:=true
```

- 시뮬레이션 / 실제 로봇 환경에 따라 노드 실행 여부 결정 가능

- `IfCondition`: 값이 참(`True`)일 때만 실행
- `UnlessCondition`: 값이 거짓(`False`)일 때만 실행
- `Node()` 의 `condition` 파라미터에 할당하여 사용

```
from launch.conditions import IfCondition
from launch.substitutions import LaunchConfiguration

DeclareLaunchArgument('use_sim_time', default_value='false'),
Node(
    package='rviz2',
    executable='rviz2',
    condition=IfCondition(LaunchConfiguration('use_sim_time'))
    # use_sim_time:=true 일 때만 rviz2 실행
)
```

Part 3. 파라미터(Parameter) 관리 실전

- 코드 수정·재컴파일 없이 노드 동작을 실시간으로 변경하는 핵심 기능
 - 시나리오: 실내 → 실외 환경 전환
 - 센서 민감도, 속도 제한 값을 즉시 변경해야 하는 상황
 - 코드를 건드리지 않고 파라미터 값만 바꿔서 해결
 - 시나리오: 여러 대의 로봇 운영
 - 서로 다른 카메라 해상도를 가진 로봇 A, B, C
 - 동일한 코드 + 각 로봇 전용 YAML 파일만 교체하여 운영
 - 핵심 장점
 - 재배포 불필요 → 현장 유연 대응
 - YAML 파일로 설정을 버전 관리 가능

- Python(rclpy) 노드에서 파라미터 사용 시 반드시 먼저 선언(declare)해야 함
 - `declare_parameter(name, default_value)`
 - 파라미터 이름과 기본값을 정의
 - 기본값의 타입으로 파라미터 타입이 자동 결정
 - `get_parameter(name).value`
 - 선언된 파라미터 값을 가져옴
 - `.value` 속성으로 실제 값에 접근 (`.value` 생략 시 파라미터 객체 반환)

```
# 선언 → 읽기 순서로 사용
self.declare_parameter('publish_rate', 15.0) # 선언 (기본값: 15.0)
self.rate = self.get_parameter('publish_rate').value # 값 읽기
```

- ROS2 파라미터는 다음과 같은 데이터 타입을 지원
 - Boolean — 참/거짓: `True` / `False`
 - Integer — 정수: `5`, `100`, `-3`
 - Double (Float) — 부동 소수점: `3.14`, `0.001`
 - String — 문자열: `"camera/image_raw"`, `"/dev/video0"`
 - List (Array) — 위 타입들의 배열: `[640, 480]`, `['a', 'b']`

```
self.declare_parameter('use_sim', False)      # Boolean
self.declare_parameter('max_speed', 2.0)      # Double
self.declare_parameter('queue_size', 10)      # Integer
self.declare_parameter('topic_name', 'image_raw') # String
self.declare_parameter('image_size', [640, 480]) # List
```

YAML 파라미터 파일 작성법

- 파라미터가 많을 때는 YAML 파일로 외부 관리
 - 필수 구조: 노드 이름 → `ros__parameters` 계층 (언더스코어 2개!)

```
# config/camera_params.yaml
my_camera_node:      # 실행 노드 이름과 반드시 일치
  ros__parameters:    # ← 이 키워드를 반드시 써야 함 (언더스코어 2개)
    publish_rate: 30.0
    topic_name: "camera/image_raw"
    image_size: [640, 480]
```

- 다중 노드 파라미터를 하나의 YAML에 작성 가능

```
camera_node:
  ros__parameters:
    fps: 30
lidar_node:
  ros__parameters:
    range: 10
```

런치 파일에서 YAML 파라미터 파일 로드

- `Node()` 액션의 `parameters` 인자에 YAML 파일 경로를 전달

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    # 패키지 설치 경로에서 YAML 파일 경로 계산
    config = os.path.join(
        get_package_share_directory('my_camera_pkg'),
        'config',
        'camera_params.yaml'
    )

    node = Node(
        package='my_camera_pkg',
        executable='camera_node_exe',
        parameters=[config]    # YAML 파일 경로를 리스트로 전달
    )

    return LaunchDescription([node])
```

- 노드가 실행 중일 때도 파라미터를 변경하고 즉시 반영 가능

- `ros2 param list`

- 현재 실행 중인 모든 노드의 파라미터 목록 확인

```
ros2 param list
```

- `ros2 param get <노드명> <파라미터명>`

- 특정 파라미터의 타입과 현재 값 출력

```
ros2 param get /camera_node publish_rate
```

- `ros2 param set <노드명> <파라미터명> <값>`

- 실행 중인 노드의 파라미터를 즉시 변경

```
ros2 param set /camera_node publish_rate 15.0
```

- 파라미터가 외부에서 변경될 때 노드 내부에서 즉시 반응하는 콜백

```
from rcl_interfaces.msg import SetParametersResult

class CameraNode(Node):
    def __init__(self):
        super().__init__('my_camera_node')
        self.declare_parameter('publish_rate', 15.0)
        self.rate = self.get_parameter('publish_rate').value

        # 파라미터 변경 시 호출될 콜백 등록
        self.add_on_set_parameters_callback(self.parameter_callback)

    def parameter_callback(self, params):
        for param in params:
            if param.name == 'publish_rate':
                self.rate = param.value
                self.get_logger().info(f'주기 변경됨: {self.rate}Hz')
        return SetParametersResult(successful=True)
```

- `ros2 param set` 으로 값을 바꾸면 콜백이 즉시 실행됨

(실습) 카메라 노드 전체 예제 코드

```
import rclpy
from rclpy.node import Node
from rcl_interfaces.msg import SetParametersResult

class CameraNode(Node):
    def __init__(self):
        super().__init__('my_camera_node')

        # 1. 파라미터 선언 (이름, 기본값)
        self.declare_parameter('publish_rate', 10.0)
        self.declare_parameter('topic_name', 'image_raw')
        self.declare_parameter('image_size', [640, 480])

        # 2. 파라미터 값 가져오기
        self.rate = self.get_parameter('publish_rate').value
        self.topic = self.get_parameter('topic_name').value
        self.size = self.get_parameter('image_size').value

        self.get_logger().info(
            f'카메라 시작: {self.topic} ({self.size[0]}x{self.size[1]}) @ {self.rate}Hz')

        # 3. 파라미터 변경 콜백 등록
        self.add_on_set_parameters_callback(self.parameter_callback)

        # 1. 퍼블리셔 생성:
        self.publisher_ = self.create_publisher(Image, self.topic, 10)
        # 2. 타이머 설정: timer_callback 실행
        self.timer = self.create_timer(1 / self.rate, self.timer_callback)
        # 3. 웹캠 연결 (0번 카메라)
        self.cap = cv2.VideoCapture(0)
        self.bridge = CvBridge()
```


(실습) 카메라 노드 전체 예제 코드

```
def parameter_callback(self, params):
    for param in params:
        if param.name == 'publish_rate':
            self.rate = param.value

            self.timer.cancel()
            self.timer = self.create_timer(1.0 / self.rate, self.timer_callback)

            self.get_logger().info(f'주기 변경됨: {self.rate}Hz')
    return SetParametersResult(successful=True)

def timer_callback(self):
    ret, frame = self.cap.read()
    if ret:
        # OpenCV 이미지를 ROS2 이미지 메시지로 변환하여 퍼블리시
        resized = cv2.resize(frame, self.size)
        img_msg = self.bridge.cv2_to_imgmsg(resized, encoding="bgr8")
        self.publisher_.publish(img_msg)
        self.get_logger().info('이미지 발행 중...')

def main(args=None):
    rclpy.init(args=args)
    node = CameraNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    rclpy.shutdown()
```

Part 4. 심화 실습 — 동적 파라미터 & YAML 연동



- 두 가지 실습을 통해 실전 파라미터 관리 완성
 - 실습 1 — YAML 파일 연동
 - `camera_params.yaml` 을 작성하여 런치 파일에서 자동 로드
 - 노드 실행 시 기본값을 YAML에서 읽어오고, 런타임에 `ros2 param set` 으로 오버라이드
 - 실습 2 — 동적 `image_size` 변경
 - `ros2 param set /my_camera_node image_size "[320, 240]"` 명령이 **실시간으로 반영**되도록 노드 수정
 - 파라미터 콜백에서 OpenCV `cv2.resize()` 해상도를 동적으로 변경